EE675: Introduction to Reinforcement Learning

Assignment 1 - Question 1

Rudransh Verma (230881)

April 13, 2026

1 Working Principles

This section details the mathematical modeling of the predator-prey grid world as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

1.1 Environment Dynamics

- **Simulator**: Implements the generative transition $s_{t+1} \sim P(\cdot|s_t, a_t)$. The predator moves deterministically, while the prey follows a uniform random walk over its valid neighborhood. If a catch occurs, the prey respawns uniformly in any cell not occupied by the predator.
- **Transition Kernel**: A matrix $\mathbf{P} \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|\times|\mathcal{S}|}$ representing $P(s'|s, a)$. It is computed as the product of the predator's deterministic movement and the prey's transition probabilities.
- **Reward Function**: Since the prey moves stochastically, $R(s, a)$ is the expectation $\mathbb{E}[r|s, a]$, calculated as the probability that the predator's chosen move coincides with the prey's random move.

1.2 Policy and Evaluation

- **Sample Policy**: Implements a stationary stochastic policy $\pi(a|s)$ which allocates 50% probability to actions minimizing Manhattan distance to the prey and 50% to uniform exploration.
- **Policy Induction**: The kernel and reward are induced by the policy such that $P^\pi = \sum \pi P$ and $R^\pi = \sum \pi R$, reducing the MDP to a Markov Reward Process (MRP).
- **Value Functions**: State-values V^π are solved via the Bellman Expectation Equation. Action-values $Q^\pi(s, a)$ are then derived by projecting V^π through the transition kernel.

2 Optimization Strategy and Implementation Notes

Due to the $O(N^4)$ growth of the state space, the following optimizations were implemented to ensure computational tractability for $N = 12$.

2.1 Iterative Policy Evaluation

As specified in the assignment, we avoid matrix inversion, which scales as $O(|\mathcal{S}|^3)$. Instead, we use iterative evaluation. By repeatedly applying the Bellman operator $T^\pi V = R^\pi + \gamma P^\pi V$, the value function converges to its fixed point. This method is significantly more efficient as it leverages the sparsity of the transition kernel.

2.2 Sparse Matrix Operations

All kernels are stored and manipulated using `scipy.sparse` CSR formats. This ensures that memory consumption remains linear relative to the number of valid transitions rather than the square of the state space, which is critical for handling the $20,736$ states present at $N = 12$.

3 Results

3.1 Initial State Value and Runtime Analysis

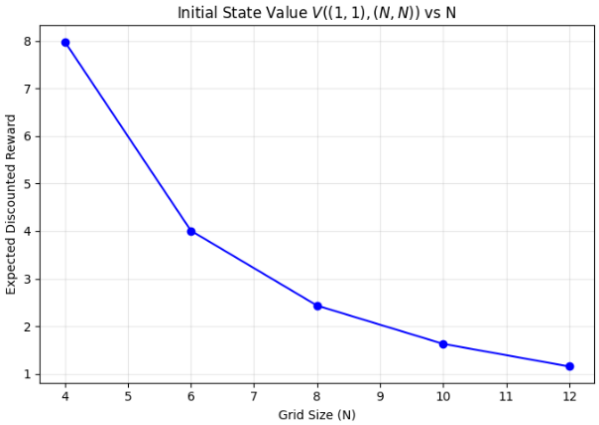


Figure 1: Value of initial state vs N.

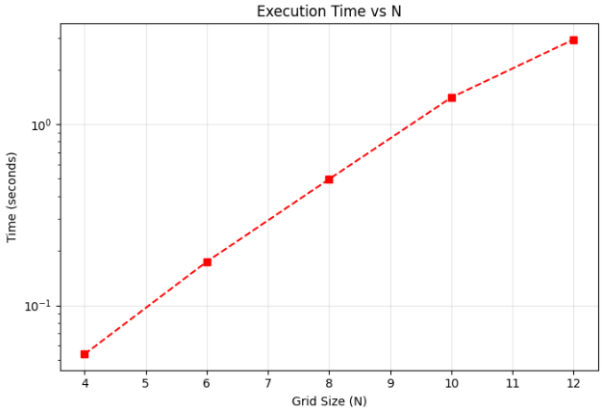


Figure 2: Run time vs N.

As the grid size N increases, the value of the initial state $((1,1),(N,N))$ decreases due to the larger Manhattan distance and the $\gamma = 0.99$ discount factor. The runtime plot confirms that despite the exponential growth in states, sparse iterative methods maintain manageable execution times.